

Analyzing Non-diagonal Preconditioning Optimizers in Deep Learning

Bingran Li 122020087

December 4, 2024

1 Introduction

Non-diagonal preconditioning method is not widely used due to expensive computation and memory costs. Recently there are some new optimizers using diagonal preconditioner, such as Adam-mini (Zhang, Chen, Li, et al., 2024), and some using non-diagonal preconditioner, such as Shampoo (Gupta et al., 2018), SOAP (Vyas et al., 2024). In this paper, we compare the performance of those optimizers on some simple models, such as linear model, MLP, and Transformer to analyze their pros and cons. To do this, we design SOAP-mini, a variant of SOAP, which takes layerwise average of second-order estimation on the eigenspace provided by Shampoo’s preconditioner. We use SOAP-mini to see the effect of different adaptive learning rate schedule in the eigenspace. We find that there is a trade-off between reducing memory usage by approximation of Adam’s and Adagrad’s preconditioners (adaptive learning rate schedule) and instability of the algorithms due to approximation. And we also find that SOAP is stronger to avoid bad local minima than others. Based on our experiment results, we propose a modification of our optimizer SOAP-mini, which run Adam-mini on the eigenspace.

2 Background

In supervised learning, we want to predict an output $y \in \mathbb{R}^{d_y}$ given an input $x \in \mathbb{R}^{d_x}$, based on our observation dataset $D := \{(x_i, y_i)\}$. We define a parameterized prediction function $h_w(\cdot) : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_y}$ by parameter vector $w \in \mathcal{W} \subseteq \mathbb{R}^d$. Our objective is to find the prediction function in the family $\mathcal{H} := \{h_w : w \in \mathcal{W}\}$ that minimizes the losses incurred from inaccurate prediction. Define a loss function $l : \mathbb{R}^{d_y} \times \mathbb{R}^{d_y} \rightarrow \mathbb{R}$, such that the loss of our prediction $h_w(x)$ with the ground truth y is $l(h_w(x), y)$.

Our objective function to minimize is the average loss of choosing parameter w (or we call it expected risk) $f(w) := \mathbb{E}[l(h_w(x), y)]$, where x and y are random variables and follow an unknown joint distribution dP . However, since we don’t know dP , we instead use a batch $B := \{(x_i, y_i)\}_{i=1}^n \subseteq D$ to calculate an estimator $f_B(w)$, which is called empirical risk, defined by $f_B(w) := \frac{1}{n} \sum_{i=1}^n l(h_w(x_i), y_i)$. An optimizer apply an iterative rule beginning with an initial point w_0 to approximate the minimum w^* .

Consider the prediction function parameterized by deep neural network with parameter matrix $W^{(1)}, \dots, W^{(n)}$, with $W^{(i)} \in \mathbb{R}^{m_i \times n_i}$. $\text{vec}(W) = (W_1^T, \dots, W_m^T)^T$ is the vectorized parameter matrix, where W_i^T is the i -th row of W and $\text{vec}(\cdot)$ is the row-wise vectorization operator. This vectorization is different from the common used one, which is column-wise. It makes huge difference in the following discussion concerning Kronecker product. The concatenation of vectorized parameters is $w = (\text{vec}(W^{(1)})^T, \dots, \text{vec}(W^{(n)})^T)^T \in \mathbb{R}^d$, $d = \sum_{i=1}^n m_i n_i$. $G_t^{(i)} := \nabla f_t(W^{(i)})$, g_t is vectorized stochastic gradient.

To simplify, the stochastic optimization problem above can be reformulated by online learning framework in some case (Cesa-Bianchi et al., 2004). On round t , a learner choose w_t , then suffer a loss $f_t(w_t)$ as feedback. In our context, f_t is given by f_{B_t} evaluated over a batch of data B_t . The goal of the learner is to minimize its regret, defined by

$$R_T = \sum_{t=1}^T f_t(w_t) - \min_{w \in \mathcal{W}} \sum_{t=1}^T f_t(w). \quad (1)$$

3 Diagonal Preconditioning

3.1 Adagrad

Adagrad (Ward et al., 2020) originate as an online learning algorithm and use adaptive step size containing second-order information. It maintain a precondition matrix $P = V_t^{-\frac{1}{2}} \in \mathbb{R}^{d \times d}$, a sum of outer product of stochastic gradient vector

$$V_t := \begin{cases} \sum_{i=1}^t g_i g_i^T & \text{(full-matrix Adagrad)} \\ \sum_{i=1}^t \text{diag}(g_i^{\odot 2}) & \text{(diagonal Adagrad)}. \end{cases} \quad (2)$$

The update of parameters with stepsize or learning rate $\eta_t > 0$ is

$$w_{t+1} = w_t - \eta_t V_t^{-\frac{1}{2}} g_t, \quad (3)$$

Remark: In the original paper of Shampoo and SOAP (Gupta et al., 2018; Vyas et al., 2024), they deal with an iteration for a single parameter matrix $W^{(i)}$. So the update of preconditioner V_t , vectorized weights w_t and stochastic gradient g_t in the above iteration are replaced by layer-wise update $g_t^{(i)} := \text{vec}(G_t^{(i)})$, $V_t^{(i)}$, and $w_t^{(i)} := \text{vec}(W^{(i)})$ for each layer. For diagonal Adagrad, the two update rule makes no difference. For full-matrix Adagrad, however, the preconditioner in the whole iteration describe in Shampoo and SOAP is

$$V_t = \text{blkdiag}(V_t^{(1)}, \dots, V_t^{(n)}) = \begin{pmatrix} V_t^{(1)} & 0_{m_1 n_1 \times m_2 n_2} & \cdots & 0_{m_1 n_1 \times m_n n_n} \\ 0_{m_2 n_2 \times m_1 n_1} & V_t^{(2)} & \cdots & 0_{m_2 n_2 \times m_n n_n} \\ \vdots & \vdots & \ddots & \vdots \\ 0_{m_n n_n \times m_1 n_1} & 0_{m_n n_n \times m_2 n_2} & \cdots & V_t^{(n)} \end{pmatrix}, \quad (4)$$

which differs from full-matrix Adagrad with sum of outer product of the whole stochastic gradient g_t .

3.2 Adam

Adam (Kingma & Ba, 2017) adopt diagonal preconditioner of Adagrad V_t with moving average inplace of the summation, and maintain an exponential moving average of the gradient g_t , denoted as m_t , from RMSprop (Hinton et al., 2012). The diagonal preconditioner V_t is symmetric positive semi-definite, a regularization term ϵ is added on each diagonal elements to keep its positive-definiteness. The update rule is given by

$$w_{t+1} = w_t - \eta_t (\sqrt{V_t} + \epsilon I)^{-\frac{1}{2}} m_t, \quad (5)$$

where square root $\sqrt{\cdot}$ is componentwise. For simplicity, we neglect the bias correction coefficient in moving average terms.

In 2-dimensional layer case $W^{(i)}$, we denote $M_t^{(i)}$ and $H_t^{(i)}$ as the moving average of the gradient matrix $G_t^{(i)}$ and componentwise square of gradient matrix $G_t^{(i)\odot 2}$, respectively. The Adam update rule of a single parameter matrix is given by,

$$W_{t+1}^{(i)} = W_t^{(i)} - \eta_t \frac{M_t^{(i)}}{\sqrt{H_t^{(i)} + \epsilon}}, \forall i = 1, \dots, n, \quad (6)$$

where division $\cdot/\cdot$ and addition $+$ are componentwise.

3.3 Adam-mini

Adam-mini (Zhang, Chen, Li, et al., 2024) reduce the memory usage of V by taking advantage of the block-diagonal structure of the hessian matrix of parameters in neural network. It partitions the parameters into blocks based on the smallest dense sub-blocks in the hessian matrix. Take query parameters in Transformer as an example, the number of partition should be equal to the number of multi-head attention. For four-head attention, the partitioned query parameters are W_1^Q , W_2^Q , W_3^Q and W_4^Q . Each of them is a single layer in Transformer. It assigns an average of the diagonal elements of the preconditioner of each block to the whole block, which reduces memory usage. Besides this, Adam-mini is the same as Adam.

3.4 Adafactor

Adam dominates other optimizers in deep learning until now. To improve Adam, notice that Adam need to maintain a momentum term and a scaling term, both of which consume d memory as large as the gradient g_t . To reduce the memory usage, Adafactor replaces $H_t^{(i)}$ by a low rank approximation of it. The approximation $H_t^{'(i)}$ is given by $(R_t C_t / 1_n^T R_t)$, where $R_t^{(i)}$ and $C_t^{(i)}$ are the moving average of the row sum and column sum of $G_t^{(i)\odot 2}$, respectively. The update rule is

$$W_{t+1}^{(i)} = W_t^{(i)} - \eta_t \frac{M_t^{(i)}}{\sqrt{H_t^{'(i)} + \epsilon}}, \forall i = 1, \dots, n. \quad (7)$$

4 Non-diagonal Preconditioning

Full-matrix Adagrad is a non-diagonal preconditioning method. But it's too expensive. There are many diagonal preconditioning optimizer other than Adam, AdamW and Adafactor, such as Lion, Sophia and Adam-mini. However, all of them do not outperform much Adam, for which we need to turn to cheap enough non-diagonal preconditioning method, Shampoo and SOAP (Gupta et al., 2018; Vyas et al., 2024).¹

4.1 Shampoo

Shampoo is a trial in non-diagonal preconditioning method. It is motivated by Adagrad, and uses Kronecker products $[L_t^{(i)}]^{1/2} \otimes [R_t^{(i)}]^{1/2} =: \tilde{V}_t^{(i)}$ to approximate layerwise full-matrix Adagrad $V_t^{(i)}$, where $L_t^{(i)}$ and $R_t^{(i)}$ are defined to be

$$L_t^{(i)} := \sum_{j=1}^t G_t^{(i)} G_t^{(i)T}, \quad R_t^{(i)} := \sum_{j=1}^t G_t^{(i)T} G_t^{(i)}. \quad (8)$$

Using the relation $\text{vec}(LGR) = (L \otimes R)\text{vec}(G)$, the update rule of Shampoo on a single layer $W^{(i)}$ can be represented by both matrix form or vectorized form as the following,

$$W_{t+1}^{(i)} = W_t^{(i)} - \eta_t [L_t^{(i)}]^{-1/4} G_t^{(i)} [R_t^{(i)}]^{-1/4}, \forall i = 1, \dots, n, \quad (9)$$

or,

$$\text{vec}(W_{t+1}^{(i)}) = \text{vec}(W_t^{(i)}) - \eta_t [\tilde{V}_t^{(i)}]^{-1/2} g_t^{(i)}, \forall i = 1, \dots, n. \quad (10)$$

The whole preconditioner of Shampoo is then $\text{blkdiag}(\tilde{V}_t^{(1)}, \dots, \tilde{V}_t^{(n)})$, which is a non-diagonal preconditioner and an approximation of layerwise full-matrix Adagrad preconditioner Equation 4.

For practical implementation of Shampoo, there are various works on it, such as reducing memory usage, distributed implementation i.e. DistributedShampoo (Shi et al., 2023), and so on. Heuristic methods are also widely investigated, such as learning rate grafting and weight decay.

4.2 SOAP

As demonstrated in SOAP (Vyas et al., 2024) page 5, claim 1, the variant of Shampoo (factors L and R with power $-1/2$ instead of $-1/4$, and so on) is equivalent to running Adafactor on the eigenbasis provided by Shampoo's preconditioner. Since Adafactor is a rank-one approximation of Adam's element-wise squared gradients $H_t^{(i)}$ for a weight matrix, it's natural to directly run Adam on Shampoo's eigenspace, which is the main idea of SOAP. Algorithm 1 is a pseudocode of a single step of SOAP for a $m \times n$ layer ignoring the red part.

To see the meaning of projected gradient G' , we write $G' = Q_L^T G Q_R$, where Q_L and Q_R are the eigenvectors of $L_t^{(i)}$ and $R_t^{(i)}$, respectively. Then denote $x' := Q_R x$ and $b' = Q_L^T b$, then the following equations are equivalent,

$$\begin{aligned} G' x' &= b' \\ \iff G x &= b. \end{aligned} \quad (11)$$

¹The following method, Shampoo and SOAP can be applied to general tensor. In this paper, we only consider the case of 2-dimensional tensor, e.g. a single layer of MLP.

So, the projection is change of basis from the original space to the eigenspace of $L_t^{(i)}$ and $R_t^{(i)}$.

Frequency of updating the preconditioner is a hyperparameter. To get eigenbasis of $L_t^{(i)}$ and $R_t^{(i)}$, SOAP use one round of power iteration method followed by `torch.linalg.qr` decomposition.

4.3 SOAP-mini

Algorithm 1 A single step of SOAP-mini for a $m \times n$ layer. We ignore the initialization and other heuristic implementation detail. Hyperparameters: learning rate η , betas(β_1, β_2), epsilon ϵ , and preconditioning frequency f . (operation `mean( $\cdot$ )` average all elements of the matrix and replace them by the mean.)

- 1: Sample batch B_t
 - 2: $G \in \mathbb{R}^{m \times n} \leftarrow \nabla_W f_{B_t}(W_t)$
 - 3: $M \leftarrow \beta_1 M + (1 - \beta_1)G$
 - 4: $L \leftarrow \beta_2 L + (1 - \beta_2)GG^T$ ▷ update L and Q and possibly Q_L and Q_R
 - 5: $R \leftarrow \beta_2 R + (1 - \beta_2)G^T G$
 - 6: **if** $t \% f == 0$ **then**
 - 7: $Q_L \leftarrow \text{Eigenvectors}(L, Q_L)$
 - 8: $Q_R \leftarrow \text{Eigenvectors}(R, Q_R)$
 - 9: **end if**
 - 10: $G' \leftarrow Q_L^T G Q_R$
 - 11: $M' \leftarrow Q_L^T M Q_R$
 - 12: $V \leftarrow \beta_2 V + (1 - \beta_2)\text{mean}(G' \odot G')$ ▷ Now we do average on $G'^{\odot 2}$
 - 13: $N' \leftarrow \frac{M'}{\sqrt{\hat{V} + \epsilon}}$
 - 14: $N \leftarrow Q_L N' Q_R^L$ ▷ Now we have preconditioned gradient in the rotated space, we project it back to the original space
 - 15: $W \leftarrow W - \eta N$
-

We modify SOAP to SOAP-mini by taking the average of the projected gradient G' in the eigenspace provided by Shampoo’s preconditioner. It’s to help us do experiment and see the effect of different adaptive learning rate schedule in the eigenspace. The algorithm is shown in Algorithm 1.

4.4 Another View on Layerwise Preconditioning

The interaction between projected gradient G' and second-order estimation V on the projected space is not known. Bernstein and Newhouse (2024) provide a new perspective to view the role of preconditioner of Adam, Shampoo and Prodigy (Mishchenko & Defazio, 2024). However, their analysis is based on the simplicity of Adam’s betas. They argue that sign descent connects Adam, so they analyze sign descent as a surrogate of Adam. As Zhang et al. (2022) said, sign descent (Adam with $\beta_1 = \beta_2 = 0$) is quite different from Adam, in herthe sense of their convergence behavior. But their analysis of Shampoo is valuable to borrow from. They show that under some

simplification of update rule, Shampoo is the steepest descent under the spectral norm. Let's review their finding.

Consider true gradient G_t instead of a stochastic one. Denote the reduced singular value decomposition of G_t as $G_t = U_t \Sigma_t V_t^T$, and setting $L_t = G_t G_t^T$ and $R_t = G_t^T G_t$. Bernstein and Newhouse proved that: 1. without the accumulation, Shampoo makes a *semi-orthogonal* weight update:

$$\begin{aligned} W_{t+1} &= W_t - \eta_t [G_t G_t^T]^{-1/4} G_t [G_t^T G_t]^{-1/4} \\ &= W_t - \eta_t U_t V_t^T, \end{aligned} \quad (12)$$

And $U_t V_t^T$ is proved to be the *closest semi-orthogonal* matrix of G_t because it minimizes $\|A - G_t\|_F$ for all $A \in \{A \in \mathbb{R}^{m \times n} : AA^T = I_m \text{ or } A^T A = I_n\}$ when G_t is full rank. 2. Shampoo is the steepest descent in the sense that the updates $\Delta W_l = -\eta_l U_l V_l^T$, $l \in [L]$ minimize the problem:

$$\arg \min_{\Delta W_1, \dots, \Delta W_L} \left[\sum_{l=1}^L \langle G_l, \Delta W_l \rangle + \frac{\lambda}{2} \max_{l=1, \dots, L} \|\Delta W_l\|_{l_2 \rightarrow l_2}^2 \right], \quad (13)$$

where $\langle \cdot, \cdot \rangle$ is the Frobenius inner product, and $\|\cdot\|_{l_2 \rightarrow l_2}$ is the operator norm, $\lambda > 0$ is sharpness.

Here under the same assumption above, we claim that without accumulation, **running Adagrad on the eigenspace provided by Shampoo's eigenspace is asymptotically Shampoo's update** $-\eta_t U_t V_t^T$. Without loss of generality, assume $m \leq n$. Write $L_t = G_t G_t^T = U_t \Sigma_t \Sigma_t^T U_t^T =: Q_L \Lambda_L Q_L^T$ and $R_t = G_t^T G_t = V_t \Sigma_t^T \Sigma_t V_t^T =: Q_R \Lambda_R Q_R^T$. Then the projected gradient (defined in Algorithm 1, line 10) $G'_t = Q_L^T G_t Q_R = \Sigma_t = \text{diag}(\sigma_1, \dots, \sigma_m)$ is the gradient projected on the eigenspace of L_t and R_t . The preconditioner of Adagrad without accumulation is given by $V_t := G'_t \odot G'_t$. Then the preconditioned projected gradient is given by

$$P_\epsilon := \frac{G'_t}{\sqrt{V_t} + \epsilon} = \begin{pmatrix} \frac{\sigma_1}{\sigma_1 + \epsilon} & 0 & \dots & 0 \\ 0 & \frac{\sigma_2}{\sigma_2 + \epsilon} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \frac{\sigma_m}{\sigma_m + \epsilon} \end{pmatrix} \quad (14)$$

Projecting P_ϵ back to the original space, we get the update of Adagrad on the eigenspace provided by L_t and R_t :

$$-\eta_t U_t P_\epsilon V_t^T \rightarrow -\eta_t U_t I_m V_t^T = -\eta_t U_t V_t^T, \text{ as } \epsilon \rightarrow 0. \quad (15)$$

By Equation 15, we get similar claim as Vyas et al. (2024), in which under other assumptions, Shampoo is equivalent to running Adafactor on the eigenspace provided by Shampoo's preconditioner. Equipped with Bernstein and Newhouse's insight, we can see a little more about the interaction between projection and operations on the projected space. The analysis whether SOAP is the steepest descent corresponding to a particular norm by the tool provided by Bernstein and Newhouse is left for future work.

Remark: We can also see why there is a square root $\sqrt{v}$ in Adam (or Adagrad). If we remove it in update, P_ϵ does not converge to identity matrix. It's not our desired result in the sense that the claim of Shampoo as a steepest descent under the spectral norm above breaks down.

5 Comparison of Memory Usage

Consider a layer of MLP, $W^{(i)} \in \mathbb{R}^{m_i \times n_i}$ where $m \geq n$. Full-matrix Adagrad’s preconditioner V costs $m_i^2 n_i^2$ memory usage. Adam totally costs $3m_i n_i$ (for G , M and V) memory usage and Adam-mini costs m_i memory usage. DistributedShampoo totally costs

$$2m_i^2 + 2n_i^2 \text{ (for } L, R \text{ and their root inverse)} + 3m_i n_i \text{ (for } G, M \text{ and } V) \quad (16)$$

memory usage. SOAP costs the same as DistributedShampoo, with maintaining Q_L and Q_R instead of the root inverse of factors. We summarize the memory usage of different optimizers in Table 1.

Table 1: Memory usage of preconditioner for a single layer of MLP ($m_i \times n_i$).

Optimizer	Memory Usage (Total)
Adam	$3m_i n_i$
Adam-mini	$2m_i n_i + m_i$
Adagrad(full-matrix)	$m_i^2 n_i^2$
Shampoo&SOAP	$2m_i^2 + 2n_i^2 + 3m_i n_i$

For computation cost, Shampoo and SOAP is more expensive than Adam and Adam-mini. We do not give specific number because it depends on different modified implementation.

We usually do not use second-order method. It is not because second-order method is worse. It is because we can not afford it. Only when we can reduce the memory and computation of second-order method. We must use it.

6 Layerwise and Blockwise Preconditioning

Zhang, Chen, Li, et al. (2024) create an principled way based on the hessian structure, to blocking parameters to precondition, efficiently and effectively reducing memory usage and stabilizing training. Following their exploitation on hessian structure of parameters, we compare their blockwise preconditioning with the layerwise preconditioning method in Shampoo and SOAP, and the blockwise preconditioning method proposed in DistributedShampoo (see section 7.1).

We find that, each block that Adam-mini partitions into, are a subset of a layer. Hence Adam-mini does a finer partitioning than Shampoo and SOAP.

The philosophy of optimization algorithm is that, for different problem classes, we may have different optimal algorithms. Therefore, our goal is not to find the best optimizer in all cases, but to find the best optimizer for a specific problem class, or even worse, to compare the performance of different optimizers on a specific problem class.

6.1 Example1: 1-layer Linear Model

Consider a 1-layer linear model with multiple output. $y \in \mathbb{R}^m$ is the output, $x \in \mathbb{R}^n$ is the input with n features, $W \in \mathbb{R}^{m \times n}$ is the weight matrix, and $b \in \mathbb{R}^m$ is the bias. The prediction is $y = Wx + b$. The loss function is $f(W) = \frac{1}{2} \|y - y^*\|_F^2$, where y^* is the ground truth. The hessian matrix of W is $H_W = \nabla_W^2 f(W)$. Then the gradient matrix is computed as $G = \nabla_W f(W) = -(y^* - Wx)x^T$. And by the equality $\text{vec}(I_m W x x^T) = (I_m \otimes x x^T) \text{vec}(W)$, we can easily see that the hessian of W is

$$H_W = I_m \otimes x x^T = \begin{pmatrix} x x^T & 0_{n \times n} & \cdots & 0_{n \times n} \\ 0_{n \times n} & x x^T & \cdots & 0_{n \times n} \\ \vdots & \vdots & \ddots & \vdots \\ 0_{n \times n} & 0_{n \times n} & \cdots & x x^T \end{pmatrix} \in \mathbb{R}^{mn}. \quad (17)$$

From here, the hessian matrix is a block-diagonal matrix with m diagonal sub-blocks $x x^T$. The number of sub-blocks equals the number of output neurons. It also indicates the correlation between the parameters of W . $\frac{\partial^2}{\partial W_{ij} \partial W_{i'j'}} f(W) \neq 0$ if W_{ij} and $W_{i'j'}$ are in the same row, $\frac{\partial^2}{\partial W_{ij} \partial W_{i'j'}} f(W) = 0$ if W_{ij} and $W_{i'j'}$ are not in the same row.

Now we look at the partition strategy of different optimizers. Adam-mini partitions the parameters of W into m blocks, each of which is a row of W . Parameters within the same block are assigned the same learning rate because of correlation of parameters. In contrast, Shampoo and SOAP deal with the layer W as a whole, and assign a full-matrix preconditioner to the whole layer. However, it's unreasonable to do so, since the precondition of a parameter should not incorporate the information of other parameters that are not correlated with it (not in the same row). DistributedShampoo partitions the parameters of W into several b by b blocks, and assign a preconditioner to each block, which saves memory. However, it ignores the correlation between the parameters in the same row.

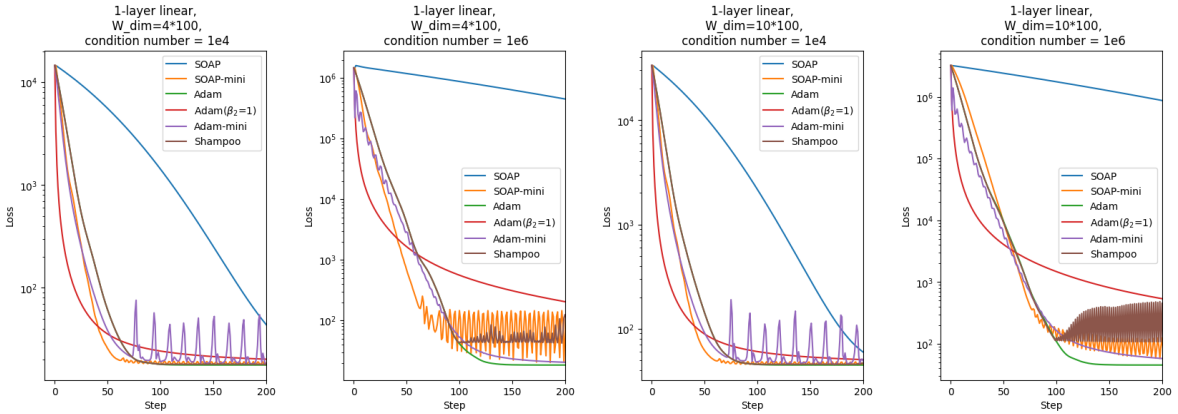


Figure 1: 1-layer linear model. We tune all optimizers. Here, Adam ($\beta_2 = 1$) is using only the second-moment information of the first iteration to assign adaptive learning rate to each of parameters

In the experiment, we compare the performance of Adam, Adam-mini, SOAP, SOAP-mini

and DistributedShampoo on the 1-layer linear model (Figure 1). We have the following interesting observations. First, SOAP-mini outperforms SOAP in all cases. Second, SOAP converges slower but stable. Third, SOAP-mini and DistributedShampoo doesn't converge when the condition number is big, and they oscillate.

More importantly, observe that Adam and SOAP are stable, while Adam-mini, SOAP-mini and DistributedShampoo are less stable. It might be because their approximation is not accurate enough. We conclude that **there is a trade-off between reducing memory usage by approximation of Adam's and Adagrad's preconditioners and instability of the algorithms due to approximation.**

We can see that SOAP is a non-diagonal preconditioner version of Adam since their behavior is similar (stable)! By analogy, can we find a non-diagonal preconditioner version of Adam-mini? We should explore this in section 7.2. The current revision, SOAP-mini is faster but not stable, because it directly takes average of projected layerwise gradient G' , which is not accurate enough. There is a trade-off between speed and stability. Compare these observations; we can adopt Adam-mini's approach to partition the parameters and subsequently assign a preconditioner provided by Shampoo or SOAP to each block. The idea of integrating Adam-mini's blocking strategy into SOAP is left for future work.

6.2 Experiment 2: 1-hidden-layer MLP

This experiment is designed to investigate the case that SOAP and SOAP-mini is applied to a MLP layer with only one row, which is also the smallest block partitioned by Adam-mini. For control group, we use a general layer with multiple rows.

We begin with blocking parameters of MLP. The Hessian structure of a MLP layer is near block-diagonal during the training (Collobert, 2004; Zhang, Chen, Ding, et al., 2024), where the elements of the hessian matrix are dense on each sub-blocks and sparse somewhere else. The number of diagonal sub-blocks equals the number of output neurons of the MLP layer.

For control group, we consider a simple case of a 1-hidden-layer MLP as binary classification. Let $y \in \mathbb{R}$ be the output, $x \in \mathbb{R}^{64}$ be the input, $W_1 \in \mathbb{R}^{10 \times 64}$ be the weight matrix of the hidden layer, $b_1 \in \mathbb{R}^{10}$ be the bias of the hidden layer, $W_2 \in \mathbb{R}^{1 \times 10}$ be the weight matrix of the output layer, and $b_2 \in \mathbb{R}$ be the bias of the output layer.

$$y = \text{Sigmoid}(W_2 \text{Tanh}(W_1 x + b_1) + b_2), \quad (18)$$

The hessian matrix is $H_{W_1} = \nabla_{W_1}^2 f(W_1)$, where $f(W_1)$ is the loss function of the MLP. Then H_{W_1} is a block-diagonal matrix with 10 diagonal sub-blocks $H_{W_1}^{(i)}$ for $i = 1, \dots, 10$. In this case (Figure 2), SOAP-mini is still faster but less stable than SOAP. There is something more interesting. First, SOAP and SOAP-mini are faster in the beginning. Second, Adam stuck in a bad local minima. **This experiment reveals the stronger ability of SOAP to avoid bad local minima than Adam** (also see the next experiment, Figure. 4).

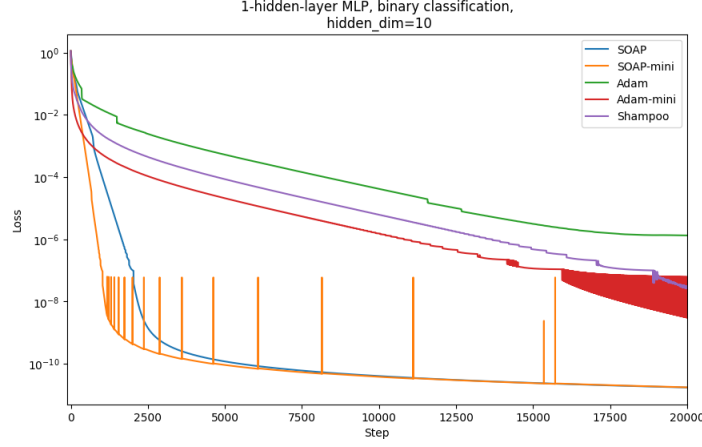


Figure 2: Control Group. (we run iterations up to $2e+5$ and Adam sticks around $1e-6$)

For treatment group, let $W_1 \in \mathbb{R}^{1 \times 1}$ and $W_2 \in \mathbb{R}^{1 \times 64}$ be row matrices. Then each layer is also a block, to which a preconditioner is applied by SOAP, SOAP-mini. We find that SOAP-mini’s performance is quite similar to Adam-mini, and faster than it in this case (Figure. 3). This result gives us a confidence to explore SOAP-mini with Adam-mini’s blocks.

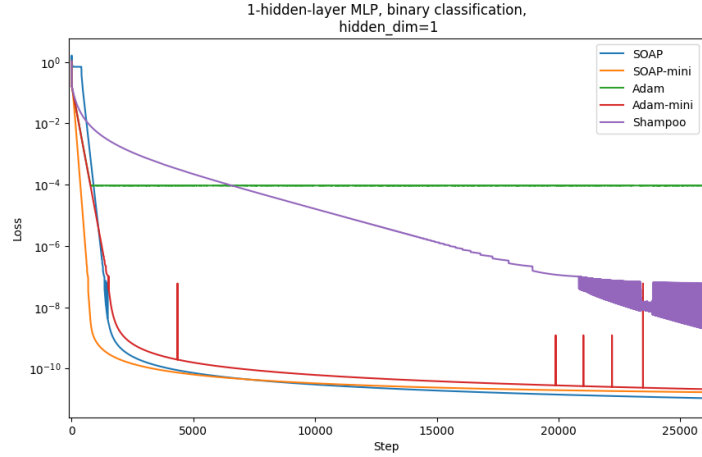


Figure 3: Treatment Group

6.3 Experiment 3: Transformer

Thanks to (Bartels, 2024), we test SOAP-mini on Transformer using model `swin_v2_s`², batch size 128 and 6,000 samples (Figure 4). Unfortunately, SOAP-mini is not stable, and is down crossed by SOAP in Transformer. It suggests one layer one learning rate on Shampoo’s eigenspace is not good, especially when each layer contains much more parameters. SOAP-mini does not make a use of the structure of the hessian matrix of parameters of a layer in Transformer, as Adam-mini does. Besides, SOAP takes more time per iteration, but it eventually surpasses Adam. The convergence rate of SOAP is quite fast. It keeps going down without getting stuck or oscillating.

²`swin_v2_s` is at https://pytorch.org/vision/main/models/generated/torchvision.models.swin_v2_s.html

Now we are ready to propose an improvement for SOAP-mini, which is to partition the parameters of a layer in Transformer into blocks, and assign a preconditioner to each block.

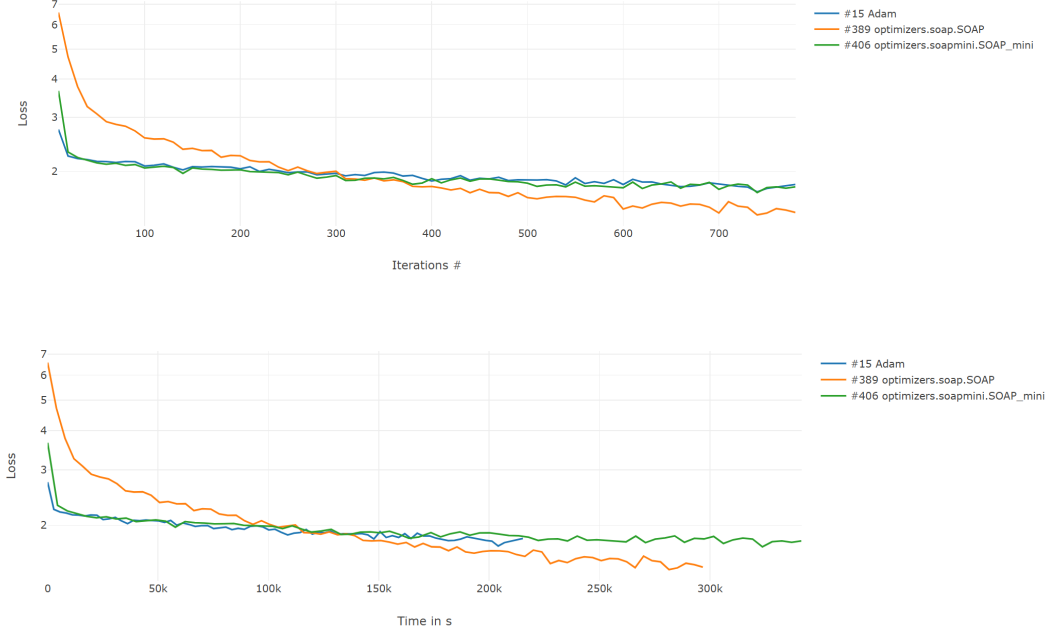


Figure 4: Tranformer. Model is `swin_v2_s`. Batch size is 128. The number of samples is 6,000

7 Improvement of SOAP-mini

7.1 Blocking Preconditioner

Blocking method in DistributedShampoo is to create sub-blocks of a layer with given block size $b \times b$ (Shi et al., 2023). Instead, we use Adam-mini’s way to do blocking. For example, for a layer of MLP $W^{(i)} \in \mathbb{R}^{m_i \times n_i}$ where $m_i \geq n_i$, we partition it into rows,

$$W^{(i)} = \begin{pmatrix} W_1^{(i)T} \\ W_2^{(i)T} \\ \vdots \\ W_m^{(i)T} \end{pmatrix} \quad (19)$$

Shampoo is then applied to each block $W_1^{(i)T}$. This also partition the preconditioner for $W^{(i)}$ to be block-diagonal. Denote $[L_j^{(i)}]^{1/2} \otimes [R_j^{(i)}]^{1/2}$ be the Shampoo’s preconditioner for $W_j^{(i)}$, then

$$\tilde{V}_{block} = \begin{pmatrix} [L_1^{(i)}]^{1/2} \otimes [R_1^{(i)}]^{1/2} & 0 & \cdots & 0 \\ 0 & [L_2^{(i)}]^{1/2} \otimes [R_2^{(i)}]^{1/2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & [L_{m_i}^{(i)}]^{1/2} \otimes [R_{m_i}^{(i)}]^{1/2} \end{pmatrix}. \quad (20)$$

SOAP-mini then runs Adam-mini on the eigenspace provided by $L_j^{(i)} \in \mathbb{R}$ and $R_j^{(i)} \in \mathbb{R}^{1 \times n_i}$ for each $W_j^{(i)}$. However, this modification increase memory usage for storing factors, since we need $m_i(1 + n_i^2)$ to store $L_j^{(i)}$ and $R_j^{(i)}$ for each block, compared to $m_i^2 + n_i^2$ of SOAP.

7.2 Blocking on Shampoo’s eigenspace

By the trade-off between reducing memory usage and instability (in section 6.1), we propose a new way to improve SOAP-mini.

As in Algorithm 1, we take $\text{mean}(\cdot)$ of $G' \odot G'$. Instead, we can take $\text{mean}(\cdot)$ of each sub-block of $G' \odot G'$, provided by Adam-mini. Again take the MLP layer as an example, we partition G' into m blocks, each of which is a row of G' . Then for each block, we take the mean of this block of $G' \odot G'$, assign the mean to this block, and update second-order estimation V on Shampoo’s eigenspace. Denote this operation as $\text{blkmean}(\cdot)$, then the line 12 of Algorithm 1 is replaced by

$$V \leftarrow \beta_2 V + (1 - \beta_2) \text{blkmean}(G' \odot G') \quad (21)$$

This saves memory usage for storing V , since we only need m_i to store V , compared to $m_i n_i$ of SOAP. It might be more stable than SOAP-mini. SOAP-mini only needs 1 to store V , which is not accurate enough.

8 Summary and Discussion

In this paper, we analyze non-diagonal preconditioning methods, relating it with Adam-mini’s partitioning strategy. We find there is a trade-off between reducing memory usage by approximation and stability. Finally, we proposed a new method to improve the SOAP-mini algorithm. We introduced a blocking operation on Shampoo’s eigenspace. This methods aim to reduce memory usage and improve stability. We believe that our proposed methods can enhance the performance of the SOAP-mini algorithm.

References

- Bartels, T. (2024). *Result viewer*. Retrieved November 18, 2024, from <https://tinko92.github.io/frontend/> (cit. on p. 10).
- Bernstein, J., & Newhouse, L. (2024). Old Optimizer, New Norm: An Anthology. <https://doi.org/10.48550/arXiv.2409.20325> (cit. on pp. 5, 6).
- Cesa-Bianchi, N., Conconi, A., & Gentile, C. (2004). On the generalization ability of on-line learning algorithms. *IEEE Transactions on Information Theory*, 50(9), 2050–2057 (cit. on p. 2).
- Collobert, R. (2004). Large scale machine learning (cit. on p. 9).
- Gupta, V., Koren, T., & Singer, Y. (2018). Shampoo: Preconditioned Stochastic Tensor Optimization. *Proceedings of the 35th International Conference on Machine Learning*, 1842–1850 (cit. on pp. 1, 2, 4).

- Hinton, G., Srivastava, N., & Swersky, K. (2012). Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. *Cited on*, 14(8), 2 (cit. on p. 3).
- Kingma, D. P., & Ba, J. (2017). Adam: A Method for Stochastic Optimization. (Cit. on p. 3).
- Mishchenko, K., & Defazio, A. (2024). Prodigy: An Expeditionally Adaptive Parameter-Free Learner. <https://doi.org/10.48550/arXiv.2306.06101> (cit. on p. 5).
- Shi, H.-J. M., Lee, T.-H., Iwasaki, S., Gallego-Posada, J., Li, Z., Rangadurai, K., Mudigere, D., & Rabbat, M. (2023). A Distributed Data-Parallel PyTorch Implementation of the Distributed Shampoo Optimizer for Training Neural Networks At-Scale. <https://doi.org/10.48550/ARXIV.2309.06497> (cit. on pp. 4, 11).
- Vyas, N., Morwani, D., Zhao, R., Shapira, I., Brandfonbrener, D., Janson, L., & Kakade, S. (2024). SOAP: Improving and Stabilizing Shampoo using Adam. (Cit. on pp. 1, 2, 4, 6).
- Ward, R., Wu, X., & Bottou, L. (2020). AdaGrad stepsizes: Sharp convergence over nonconvex landscapes. *Journal of Machine Learning Research*, 21(219), 1–30 (cit. on p. 2).
- Zhang, Y., Chen, C., Ding, T., Li, Z., Sun, R., & Luo, Z.-Q. (2024). Why Transformers Need Adam: A Hessian Perspective. <https://doi.org/10.48550/arXiv.2402.16788> (cit. on p. 9).
- Zhang, Y., Chen, C., Li, Z., Ding, T., Wu, C., Kingma, D. P., Ye, Y., Luo, Z.-Q., & Sun, R. (2024). Adam-mini: Use Fewer Learning Rates To Gain More. <https://doi.org/10.48550/arXiv.2406.16793> (cit. on pp. 1, 3, 7).
- Zhang, Y., Chen, C., Shi, N., Sun, R., & Luo, Z.-Q. (2022). Adam Can Converge Without Any Modification On Update Rules (cit. on p. 5).